

Mergesort, one pass at a time

No theorem in this exposition is mechanized. The program is concrete and the ledger is closed on the desk; every cost claim is therefore amber. The only grey box is the lower bound that this calculation does not prove.

Fix $N = 2^n$ records of w bits each. For exact constants, first take the machine-native case $w = 2^v$. The two arrays are aligned grade- $(n + v)$ subtrees under a common parent. Record r is the aligned grade- v subtree at slot r , with its key stored most-significant bit first. Thus records and their bits are both contiguous in leaf order.

Nothing asymptotic rests on the dyadic-width convention. For an arbitrary logical width w , use physical slot width $W = 2^{\lceil \log_2 w \rceil}$, store the key followed by zero padding, and compare the whole slot. Then $w \leq W < 2w$, and padding a common suffix preserves lexicographic order. Below, W denotes the physical width; in the exact case $W = w$.

1 The program

This is direct-style bottom-up mergesort. Pass j , for $j = 0, \dots, n - 1$, views the source as adjacent pairs of sorted runs, each run containing 2^j records. Two read heads start at the first records of the two runs — separated by stride $2^j W$ bits — and one write head starts at the corresponding destination slot. Compare the current records lexicographically, copy the smaller one, and advance only its read head; when one run is empty, copy the other tail. The write head advances after every record. This is the zip geometry — two monotone readers and one monotone writer — with a data-dependent choice of reader, not the oblivious alternating word of Expo 1.

The source and destination ping-pong between the two buffers. All three heads return to their buffer roots after a pass, ascend for free to the common parent, and descend into the opposite roles for the next pass. Pass zero merges singleton runs; after pass j the destination consists of sorted runs of length 2^{j+1} . After pass $n - 1$ the sorted array is in the parity-selected buffer. No final normalization copy is part of the interface.

2 A comparison is part of the pass

DESK-PROVED — not mechanized

A full-width comparison and copy have linear bit-cost

A comparison scans both grade- v record trees and returns both heads to their record roots. One head pays for the $2W - 2$ downward edges of the Euler tour and for W reads; up is free. Two heads therefore pay at most $6W - 4$. The finite control retains one of three states — equal so far, left smaller, right smaller — so branching is the result of the charged read instructions, not a hidden unit-cost operation. We deliberately finish the scan even after the order is known, making this bound data-independent and returning both heads to their roots.

Copying the chosen record pays exactly $6W - 4$: W source reads, W destination writes, and $4W - 4$ downward edges across the two record trees. This is the schedule whose correctness and unit-instruction length $10W - 8$ are mechanized as `copyWord_correct` and `copyWord_cost`; the difference is exactly its $4W - 4$ now-free up instructions. Likewise `zipWord_correct` and `zipWord_cost` mechanize the three-head streaming pattern at unit-instruction length $20W - 12$; under free-up the same word costs $12W - 6$. The merge uses those verified itineraries as building blocks, but its comparison loop and amended-price equations remain desk-level.

3 One pass

Let $P_j = \frac{N}{2^{j+1}}$ be the number of run pairs in pass j , and let q_j be the comparisons actually taken. A merge of two nonempty runs needs at most one fewer comparison than its output length, hence $q_j \leq N - P_j$ over the pass.

DESK-PROVED — not mechanized

Every merge pass costs $\Theta(NW)$

The record work is exact for the schedule above:

$$N(6W - 4) + q_{j(6W-4)}.$$

The first term copies every record once; the second scans both current records for every comparison. The remaining movement is an odometer sum at record granularity. Each input head scans the P_j merge-pair roots for $2P_j - 2$ paid descents. Within each pair it descends into its half and scans all 2^j record roots, contributing $N - P_j$ more. The two input heads therefore cost $2N + 2P_j - 4$; the output head's full record-root tour costs $2N - 2$. Thus root movement is

$$M_j = 4N + 2P_j - 6.$$

Entering the source buffer with two heads and the destination buffer with one adds exactly three paid descents; every return is free. Consequently

$$C_j = N(6W - 4) + q_{j(6W-4)} + M_j + 3,$$

$$NW \leq C_j \leq 12NW.$$

The lower bound is already forced by the NW destination writes. For unpadded dyadic records, $W = w$ and the constant is

12. For arbitrary logical width, $W < 2w$ gives

$Nw \leq C_j < 24Nw$. The stride changes only which record-root subtrees the two readers tour; its boundary spikes are precisely the $2P_j - 2$ odometer terms above, not an extra logarithm per access.

4 All passes

DESK-PROVED — not mechanized

Bottom-up mergesort costs $\Theta(N w \log N)$

There are exactly $n = \log_2 N$ passes, and ping-pong composition adds no movement beyond the three buffer-entry descents already charged in each C_j . Summing the pass bounds gives, for dyadic records,

$$Nw \log_2 N \leq C_{\text{sort}} \leq 12Nw \log_2 N.$$

For an arbitrary original width w , the padded physical width satisfies $W < 2w$, so

$$Nw \log_2 N \leq C_{\text{sort}} < 24Nw \log_2 N.$$

Hence this particular comparison mergesort has honest bit-cost $\Theta(Nw \log N)$ under free-up. The bound includes reading both operands, the comparison branch, writing the chosen record, all dyadic boundary crossings, and all buffer changes.

5 What the upper bound does not settle

UNFORMALIZED

Is the product lower bound forced by comparison sorting?

For comparison-only sorting of N distinct w -bit records into a disjoint output buffer, does every uniform dyadic-machine program have worst-case cost $\Omega(Nw \log N)$, even if it may choose a different layout and head schedule?

The classical decision-tree argument supplies $\Omega(N \log N)$ comparisons. Materializing the separate output supplies $\Omega(Nw)$ charged writes. These are two lower bounds, not their product: a bit-level comparison need not inspect all w bits, and information learned from shared prefixes might be reused. A matching theorem therefore needs a dyadic-machine argument forcing $\Omega(w)$ bit work across enough of the comparison tree, or a counterexample that exposes a smaller bound. The calculation above proves neither.

6 The third rung

Zip is one streaming word; doubling copy is a telescoping chain of streaming words. Mergesort is the first worked example whose algorithmic structure is genuinely multi-pass: locality removes a random-access logarithm inside each pass, while the algorithm's own pass count remains visible. A Lean track would define the data-dependent merge word, prove its sorted-permutation semantics, derive the closed pass equation with q_j , and compose the n passes with the ping-pong invariant. Those are exactly the amber seams above; the matching lower bound should stay grey unless a separate argument is found.